CIPA 4

GROUPE 8

Projet IA

Compte-Rendu – Romann, Maël et Florian

Table des matières

I. Introduction 2

a) Gestion de projet (Gantt) 2

II. Besoin Client 1 : Visualisation sur carte (Mael) 2

a) Contexte et objectif 2

b) Données 2

c) Variables 2

d) Prétraitement 3

e) Normalisation des données 3

f) Méthode du coude 3

g) Confirmation par métrique 3

h) Algorithmes testés 5

i) Visualisation sur carte 6

j) Script final 7

III. Besoin Client 2 : Modèle de prédiction de l'âge (Florian) 8

a) Contexte et objectif 8

b) Données 8

c) Choix et justification des variables 8

d) Prétraitement 13

e) Comparaison et choix du modèle 13

f) Optimisation par GridSearchCV 15

g) Résultats du modèle final 16

h) Conclusion 17

IV. Besoin Client 3 : Système d'alerte pour les tempêtes (Romann) 17

a) Sélection des données utiles 18

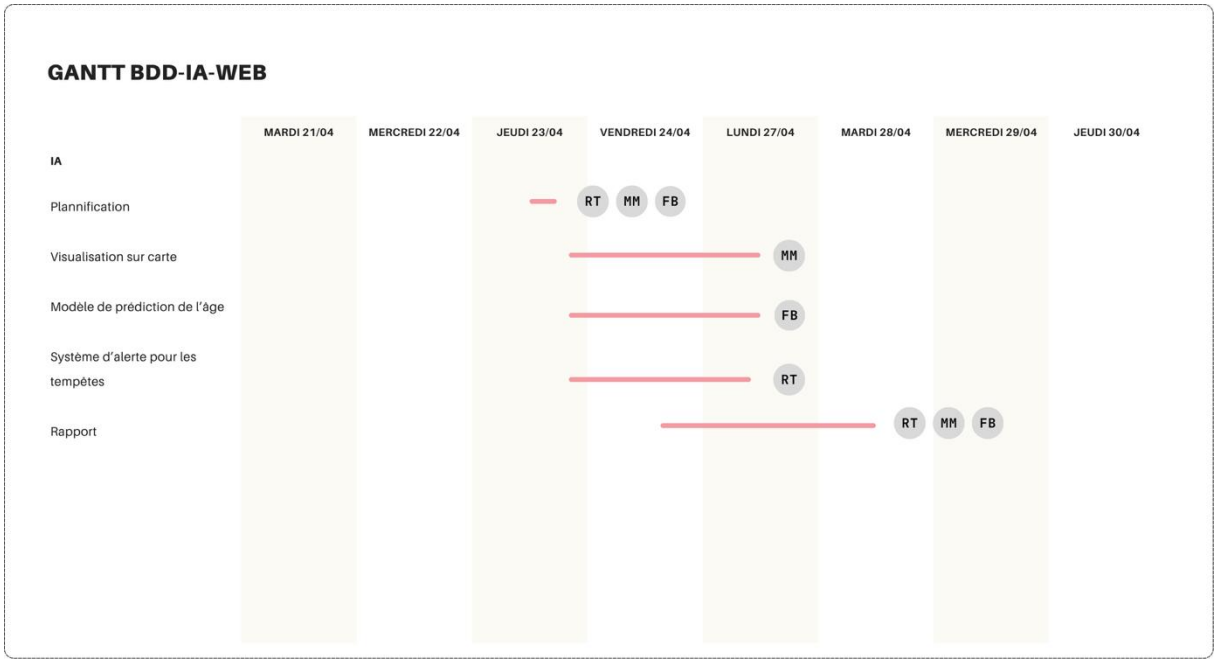
b) Définir le paramètre « alerte » 19

c) Sélection du meilleur modèle d'apprentissage 20

d) Entrainement du modèle 20

I. Introduction

a) Gestion de projet (Gantt)



II. Besoin Client 1 : Visualisation sur carte (Mael)

a) Contexte et objectif

L'objectif est de créer une visualisation sur carte qui sépare les arbres de la ville de Saint-Quentin selon leur taille, avec un nombre de catégories choisi par l'utilisateur.

Pour répondre à ce besoin, nous avons utilisé une approche d'apprentissage non supervisé (clustering) afin de regrouper automatiquement les arbres en fonction de leurs caractéristiques morphologiques, sans étiquette préalable.

b) Données

Le jeu de donnée contient 11418 lignes avant et après nettoyage des données car nous avions déjà traité les données dans la partie Big Data. Il n'y a aucune case non-renseignée.

c) Variables

Le besoin client demande de séparer les arbres selon leur taille. Nous avons donc sélectionné les trois variables qui décrivent directement la morphologie physique d'un arbre pour former les clusters :

- haut_tot (mètre) : Hauteur totale
- haut_tronc (mètre) : Hauteur du tronc uniquement
- tronc_diam (mètre) : Diamètre du tronc

Les variables sont toutes quantitatives.

d) Prétraitement

Pour vérifier la stabilité du modèle, nous séparons le jeu de donnée en 2 catégories : 80% en données d'entraînement et 20% en données de test. En apprentissage supervisé, le jeu de donnée de validation n'est pas nécessaire car il n'y a pas d'hyperparamètre à ajuster.

e) Normalisation des données

Les trois variables n'ont pas la même échelle : haut_tot peut atteindre 30 mètres, tronc_diam est en mètres avec des valeurs inférieures à 1.

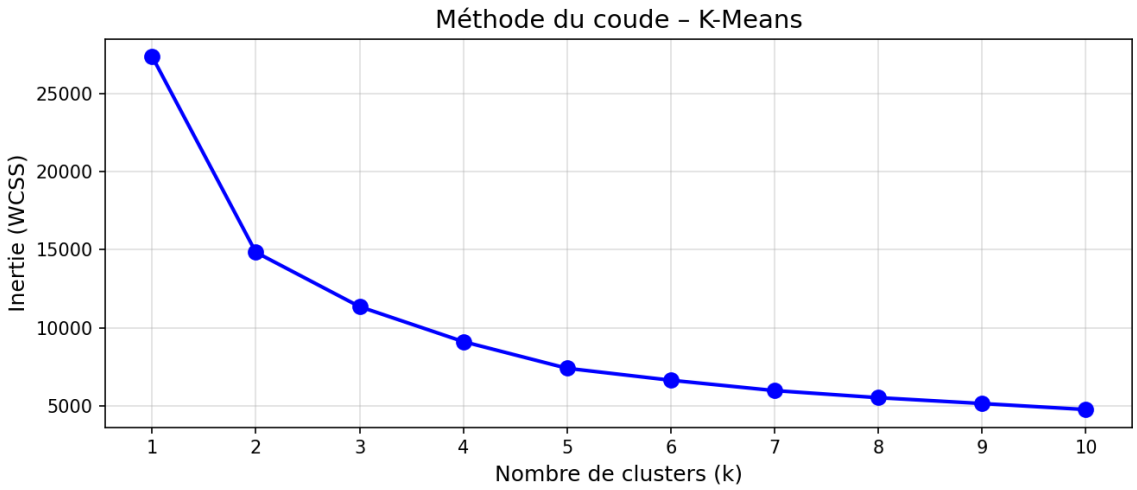
Etant donné que K-Means calcule des distances euclidiennes, une normalisation est indispensable. Nous avons appliqué un StandardScaler qui centre les données autour de 0 et les réduit à un écart-type de 1.

$X_scaled = (X - \mu) / \sigma$

f) Méthode du coude

Pour trouver le nombre optimal de cluster k, nous utilisons la méthode du coude.

Elle consiste à tracer l'inertie (somme des carrés des distances de chaque point à son centroïde) en fonction du nombre de clusters k. On cherche le point de coude où la courbe cesse de décroître fortement : ajouter un cluster supplémentaire au-delà de ce point n'apporte plus de gain significatif.



Nous avons testé k de 1 à 10. Le graphique obtenu montre un coude prononcé autour de k = 3, indiquant que 3 clusters offrent le meilleur compromis entre compacité et simplicité.

g) Confirmation par métrique

Pour confirmer le choix de k, nous avons calculé les trois métriques d'évaluation pour k allant de 2 à 6. Le k retenu est celui qui présente le meilleur Silhouette Score, le Calinski-Harabasz Index le plus élevé et le Davies-Bouldin Index le plus faible.

i. Silhouette Score

Le Silhouette Score mesure pour chaque point à quel point il est bien placé dans son cluster en combinant deux notions :

- La cohésion (a) : distance moyenne entre le point et les autres points de son propre cluster. Plus elle est faible, plus le point est bien intégré à son cluster.
- La séparation (b) : distance moyenne entre le point et tous les points du cluster voisin le plus proche. Plus elle est élevée, plus le point est bien séparé des autres clusters.

Silhouette = (b - a) / max(a, b)

La valeur est comprise entre -1 et 1. Un score proche de 1 signifie que le point est très bien classé dans son cluster. Un score proche de 0 indique que le point est à la frontière entre deux clusters. Un score négatif signifie que le point est probablement mal classé.

ii. Calinski-Harabasz

Le Calinski-Harabasz Index calcule le rapport entre la dispersion inter-cluster et la dispersion intra-cluster :

- La dispersion inter-cluster mesure à quel point les centres des clusters sont éloignés les uns des autres.
- La dispersion intra-cluster mesure à quel point les points sont proches de leur centroïde.

Plus l'index est élevé, meilleur est le clustering : les clusters sont à la fois compacts (points proches de leur centroïde) et bien séparés (centres éloignés les uns des autres).

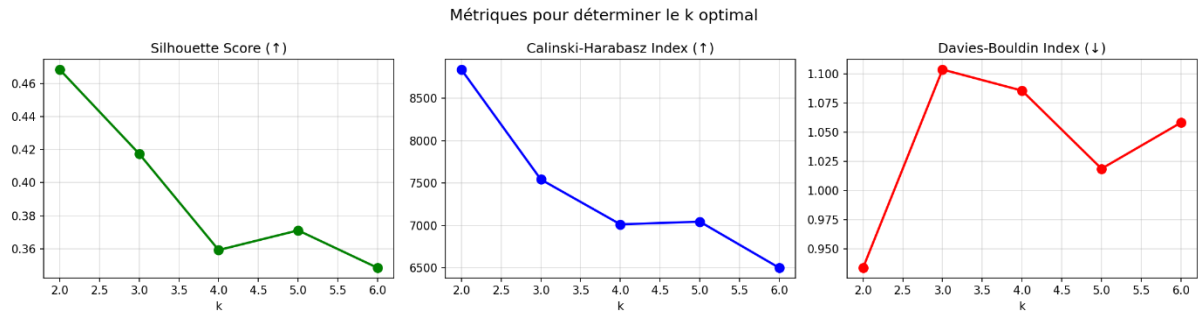
iii. Davies-Bouldin

Pour chaque cluster, le Davies-Bouldin Index calcule le rapport entre sa dispersion interne et sa distance au cluster le plus similaire, puis prend la moyenne sur tous les clusters :

Une valeur faible indique des clusters compacts et bien séparés les uns des autres.

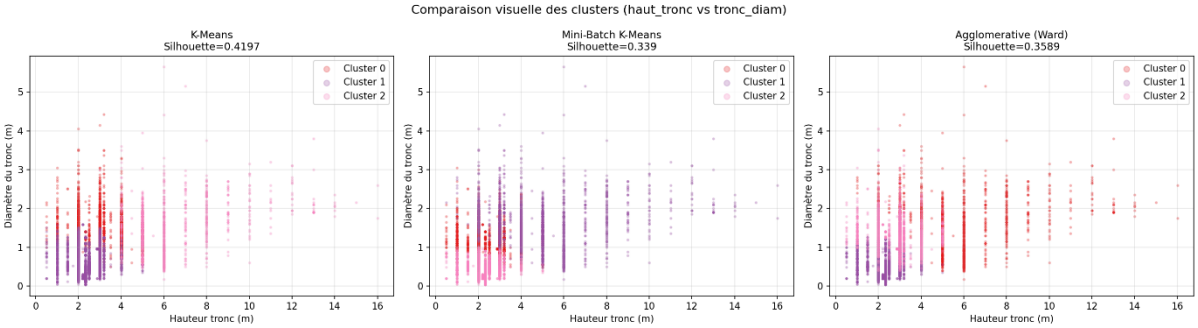
Une valeur élevée indique des clusters qui se ressemblent trop ou qui sont trop dispersés en interne.

Il est complémentaire au Silhouette Score car il évalue spécifiquement les paires de clusters les plus proches.



Les 3 métriques indiquent 3 comme le nombre de cluster le plus performant. Il est possible d'utiliser seulement 2 clusters pour les utilisateurs voulant une classification simplifiée.

h) Algorithmes testés



i. K-means

K-Means est un algorithme itératif qui cherche à minimiser la variance intra-cluster. Il fonctionne en 4 étapes :

- Initialisation : k centroïdes sont placés dans l'espace. Avec l'option k-means++, ils sont espacés intelligemment pour accélérer la convergence.
- Assignation : chaque point est affecté au centroïde le plus proche selon la distance euclidienne.
- Mise à jour : chaque centroïde est recalculé comme la moyenne de tous les points qui lui sont assignés (d'où le nom K-Means).
- Itération : les étapes 2 et 3 se répètent jusqu'à convergence (les centroïdes ne bougent plus).

Dans notre cas, les centroïdes finaux représentent les tailles moyennes de chaque groupe (Petit, Moyen, Grand)

ii. Mini-Batch K-means

Le mini-batch k-means est une variante du k-means. La logique est identique à K-Means mais au lieu d'utiliser tous les points à chaque itération, l'algorithme tire aléatoirement un sous-ensemble (batch) de données.

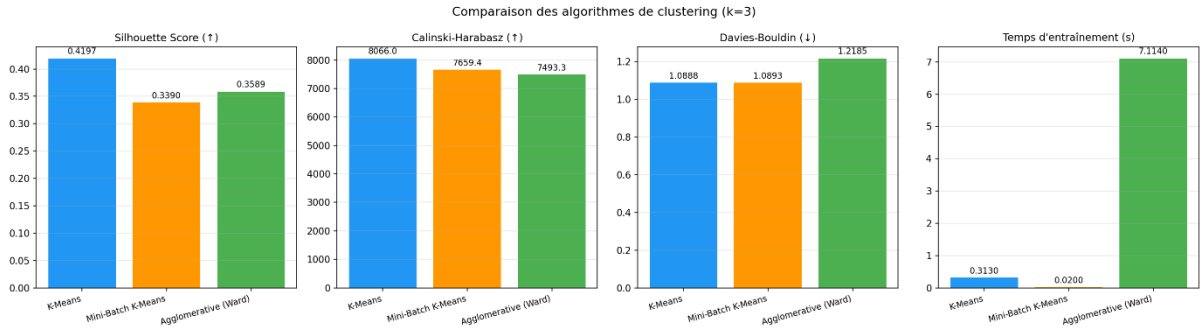
Cela rend l'algorithme significativement plus rapide sur de grands volumes de données mais les centroïdes sont légèrement moins précis.

iii. Agglomerative clustering

L’agglomerative clustering part de chaque point comme son propre cluster (n clusters au départ) et on fusionne progressivement les plus proches.

- Chaque arbre est son propre cluster → n clusters initiaux.
- On calcule la distance entre toutes les paires de clusters.
- Les deux clusters dont la fusion minimise l'augmentation de variance intra-cluster sont fusionnés.
- On répète jusqu'à atteindre k clusters.

Agglomerative Clustering ne suppose pas de forme sphérique pour les clusters, contrairement à K-Means. Il est donc plus flexible. Cependant il ne possède pas la fonction predict().

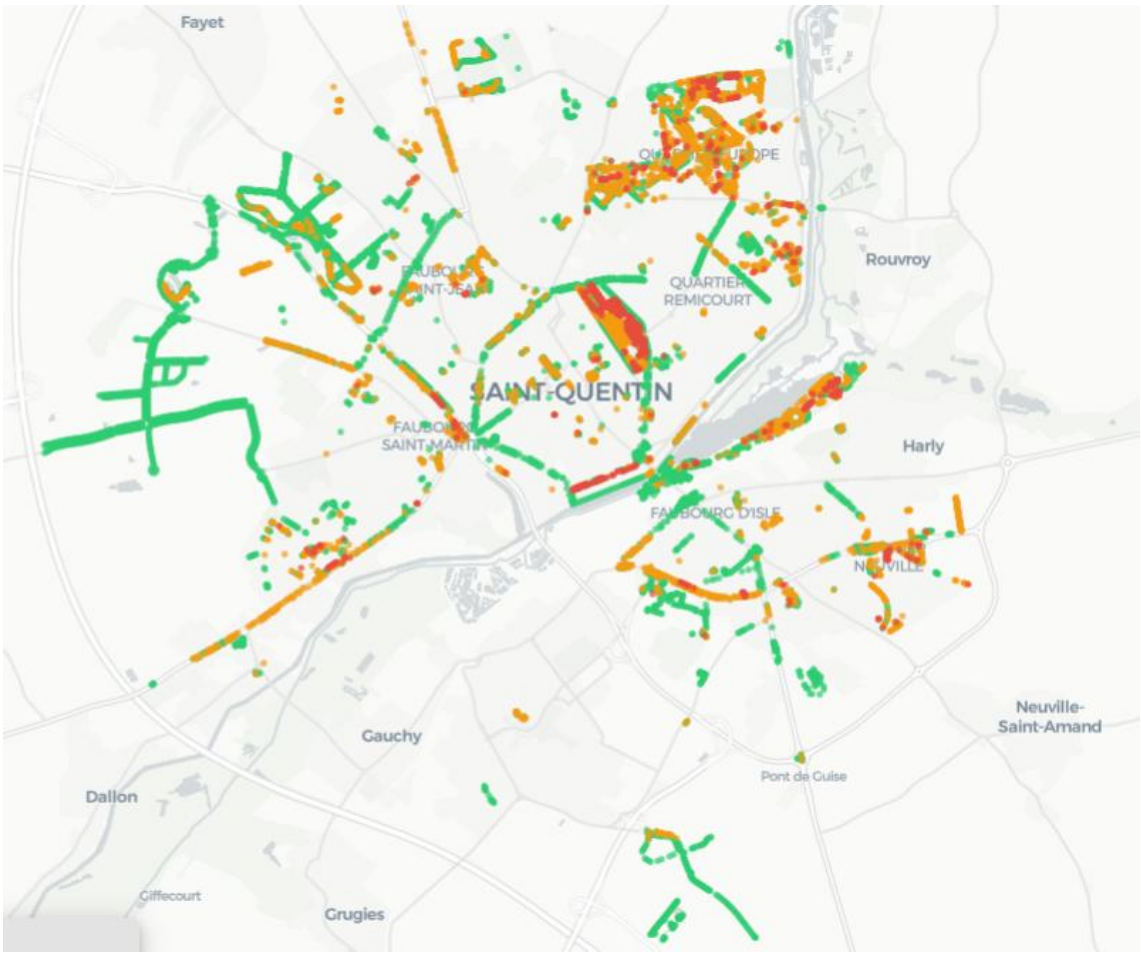


On les comparant avec les 3 métriques utilisées plus haut, on observe que l'algorithme de clustering le plus performant est le K-means, tandis que l'agglomerative est écartée dû à son caractère non-transductif (pas de fonction predict()).

i) Visualisation sur carte

La carte interactive est réalisée avec la bibliothèque Plotly sur un fond OpenStreetMap. Chaque arbre est représenté par un point coloré selon sa catégorie de taille :

- Vert : arbres Petits
- Orange : arbres Moyens
- Rouge : arbres Grands



j) Script final

Le script predict_cluster.py prend en entrée les caractéristiques d'un arbre, le nombre de cluster souhaité (2 ou 3) et renvoie sa catégorie de taille, sans relancer d'entraînement.

Il y a un fichier pkl pour k=2 et un autre pour k=3, chacun d'eux contient le modèle d'entraînement K-means, le scaler et le label map associé.

Le script charge le fichier.pkl correspondant au k choisi par l'utilisateur.

```
(.venv) isen@LAPTOP-293DF89M:~/BDD-IA-WEB/IA/1 - Visualisation-carte$ python predict_cluster.py --haut_tot 8.0 --haut_tronc 2.5 --tronc_diam 0.
30 --k 3
  warnings.warn(

========================================
  RÉSULTAT DE PRÉDICTION (k=3)
========================================
  Hauteur totale  : 8.0 m
  Hauteur tronc   : 2.5 m
  Diamètre tronc  : 0.3 m
----------------------------------------
  Cluster         : 0
  Catégorie       : Petit
========================================
```

III. Besoin Client 2 : Modèle de prédiction de l'âge (Florian)

a) Contexte et objectif

L'objectif est de prédire l'âge en années d'un arbre à partir de ses caractéristiques morphologiques mesurables sur le terrain, sans connaître son âge réel.

L'approche retenue est une régression supervisée : le modèle apprend la relation entre les mesures physiques de l'arbre (diamètre du tronc, hauteur, etc.) et la variable cible age_estim (âge estimé en années).

b) Données

Le jeu de données contient 10 308 arbres après nettoyage (suppression des valeurs manquantes et des arbres avec age_estim = 0). La variable cible age_estim varie de 1 à 200 ans, avec une moyenne de 28,4 ans et une forte concentration sur les arbres jeunes.

c) Choix et justification des variables

i. Vue d'ensemble : association de chaque variable avec l'âge

Corrélation de Pearson

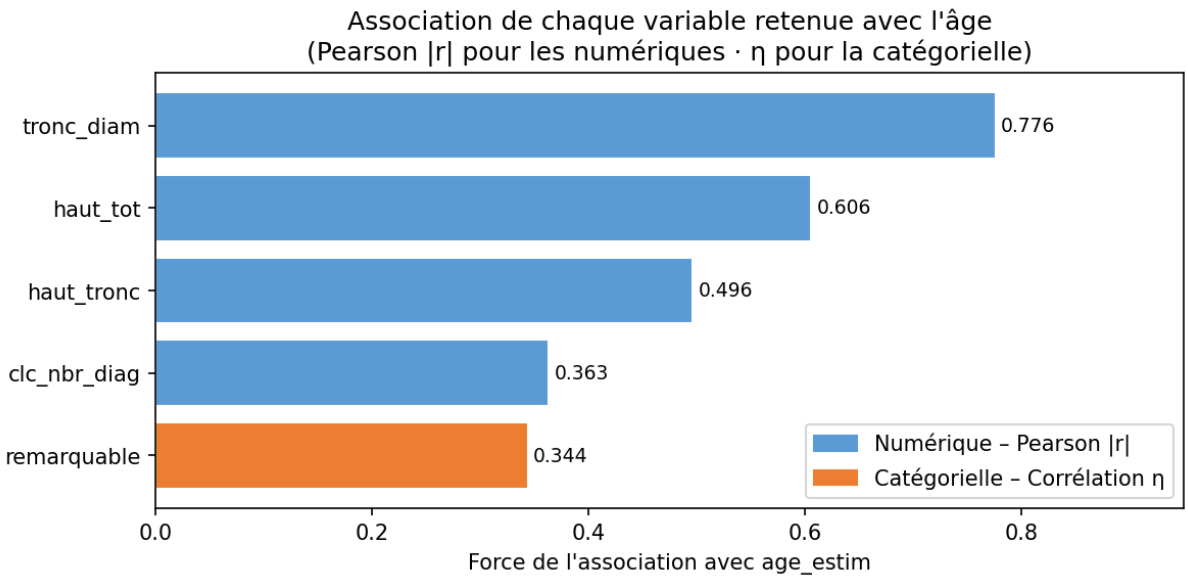
La corrélation de Pearson (notée r) mesure la force et le sens du lien linéaire entre deux variables numériques. Elle répond à la question : quand une variable augmente, est-ce que l'autre a tendance à augmenter ou à diminuer, et dans quelle mesure ?

r varie entre −1 et 1 :

- r proche de 1 : forte relation positive : quand le diamètre du tronc augmente, l'âge augmente aussi.
- r proche de −1 : forte relation négative : quand une variable augmente, l'autre diminue.
- r proche de 0 : pas de relation linéaire détectable.

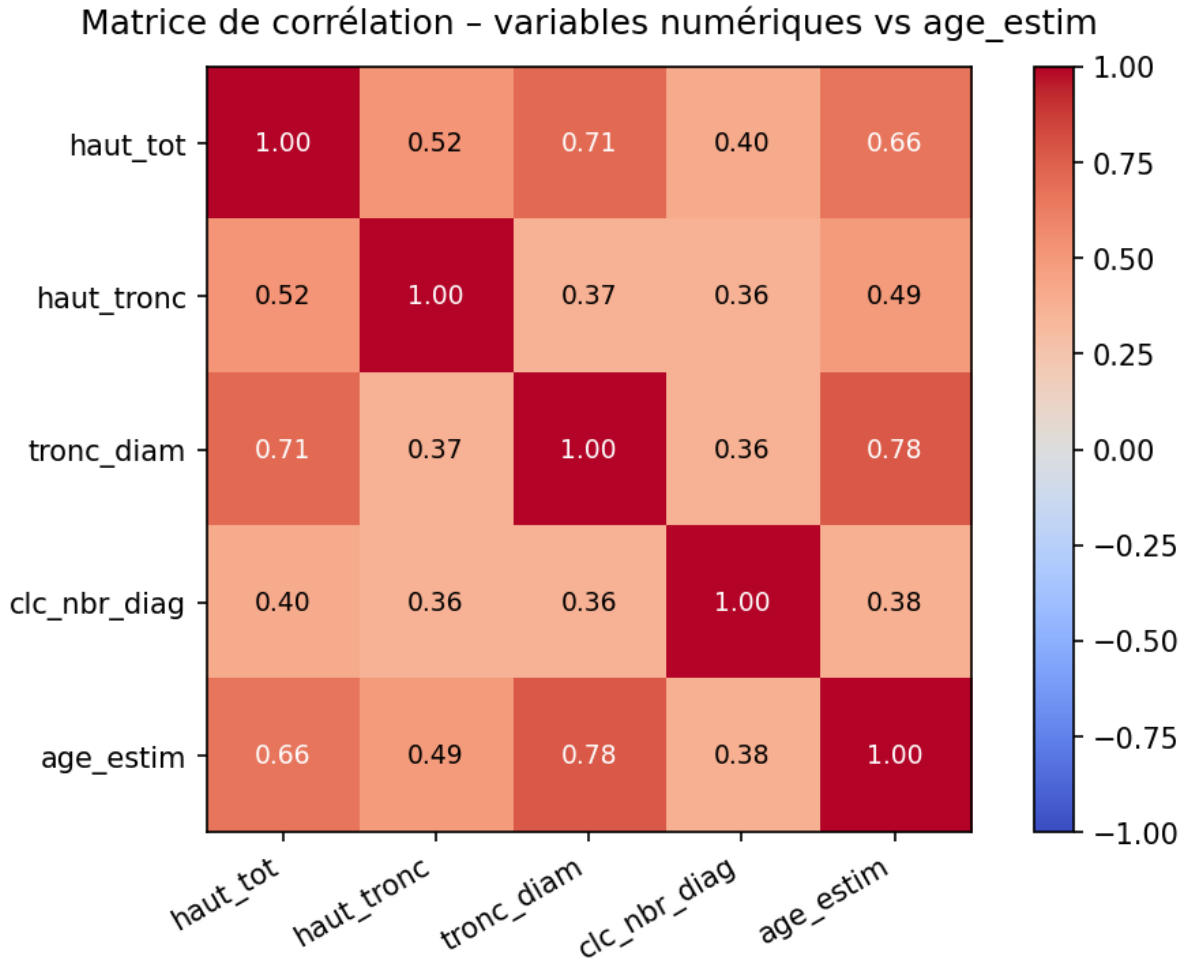
Deux limites importantes : Pearson ne détecte que les relations linéaires (une relation en courbe peut donner r ≈ 0 même si le lien est fort), et il est sensible aux valeurs extrêmes. C'est pourquoi on le complète toujours par un nuage de points.

La corrélation de Pearson ne s'applique pas aux variables catégorielles. Pour celles-ci, on utilise le rapport de corrélation η (eta). η mesure dans quelle mesure les groupes d'une variable catégorielle ont des âges moyens différents les uns des autres, si tous les groupes ont le même âge moyen, η = 0 (la variable n'apporte rien) ; si les groupes sont très différenciés, η se rapproche de 1. Comme r, η varie entre 0 et 1 et s'interprète sur la même échelle.



Ce graphique synthétise l'association de chaque variable avec age_estim sur une échelle commune. Les barres bleues représentent le |r| de Pearson (variables numériques), les barres oranges le η (variables catégorielles).

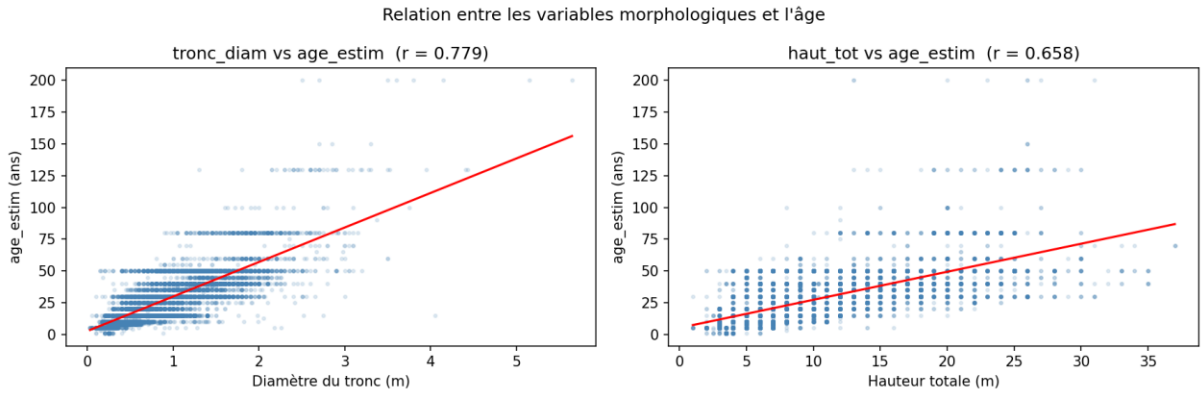
ii. Variables numériques / analyse de corrélation



La matrice de corrélation révèle plusieurs informations clés :

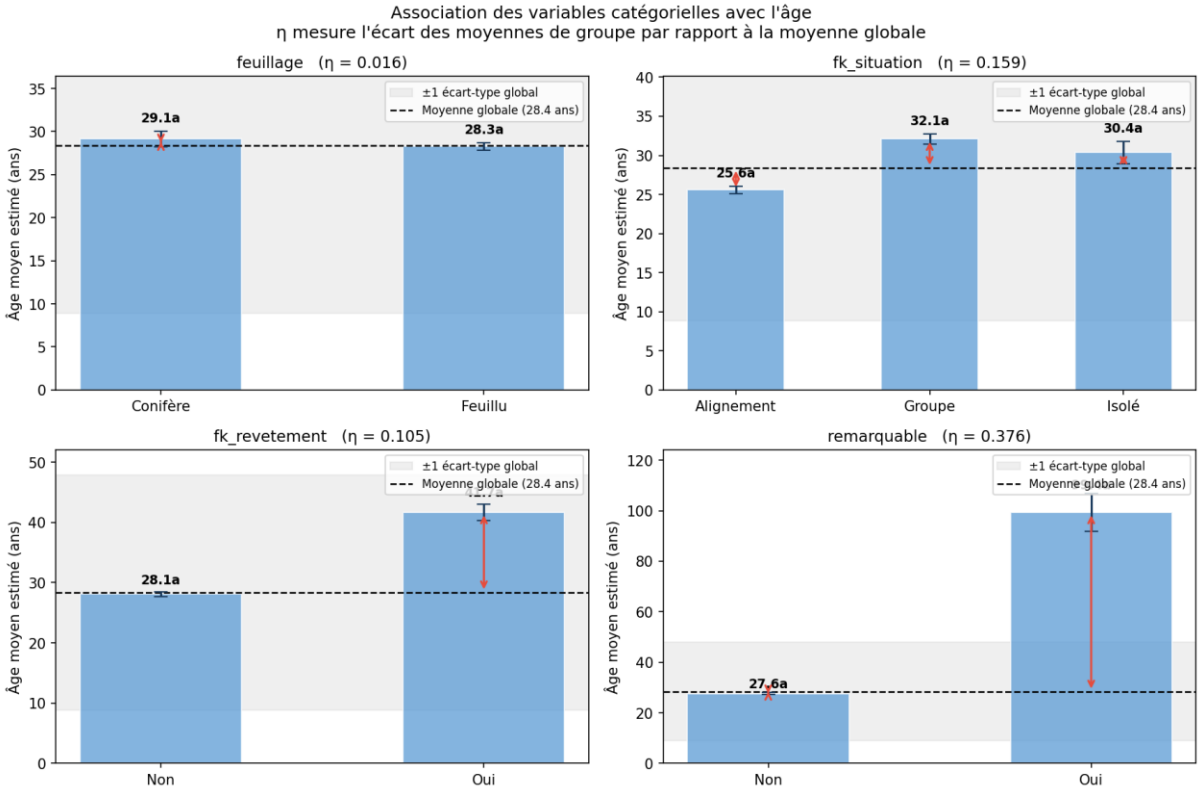
- tronc_diam est la variable la plus corrélée avec l'âge (r = 0,779). Cette relation est biologiquement attendue : le diamètre d'un arbre croît de façon quasi-continue tout au long de sa vie.
- haut_tot présente également une forte corrélation (r = 0,658). La hauteur totale augmente avec l'âge, même si elle plafonne pour certaines espèces.
- haut_tronc et clc_nbr_diag montrent des corrélations plus modérées (0,489 et 0,380), mais apportent une information complémentaire aux deux premières.
- Les corrélations entre variables explicatives (multi-colinéarité) sont également visibles : tronc_diam et haut_tot sont elles-mêmes corrélées (r ≈ 0,70). Un modèle linéaire simple serait pénalisé par cette multi-colinéarité, ce qui renforce l'intérêt d'un modèle à base d'arbres.

iii. Relation visuelle entre les variables clés et l'âge



Ces nuages de points confirment la relation positive entre les mesures morphologiques et l'âge. La droite de régression linéaire (en rouge) illustre la tendance générale.

iv. *Variables catégorielles rapport de corrélation η*



Ce graphique rend η directement lisible : chaque barre représente la moyenne d'âge du groupe, la ligne pointillée noire est la moyenne globale, et la flèche rouge indique l'écart entre les deux. La zone grise représente ±1 écart-type global. Plus les barres s'éloignent de la ligne pointillée, plus η est élevé.

L'analyse de chaque variable catégorielle conduit aux décisions suivantes :

remarquable (η = 0,376) retenue. Les arbres non-remarquables ont une moyenne de 27,6 ans (proche de la moyenne globale), tandis que les arbres remarquables affichent une moyenne de près de 100 ans. L'écart est massif et la flèche rouge très longue : cette variable est clairement informative.

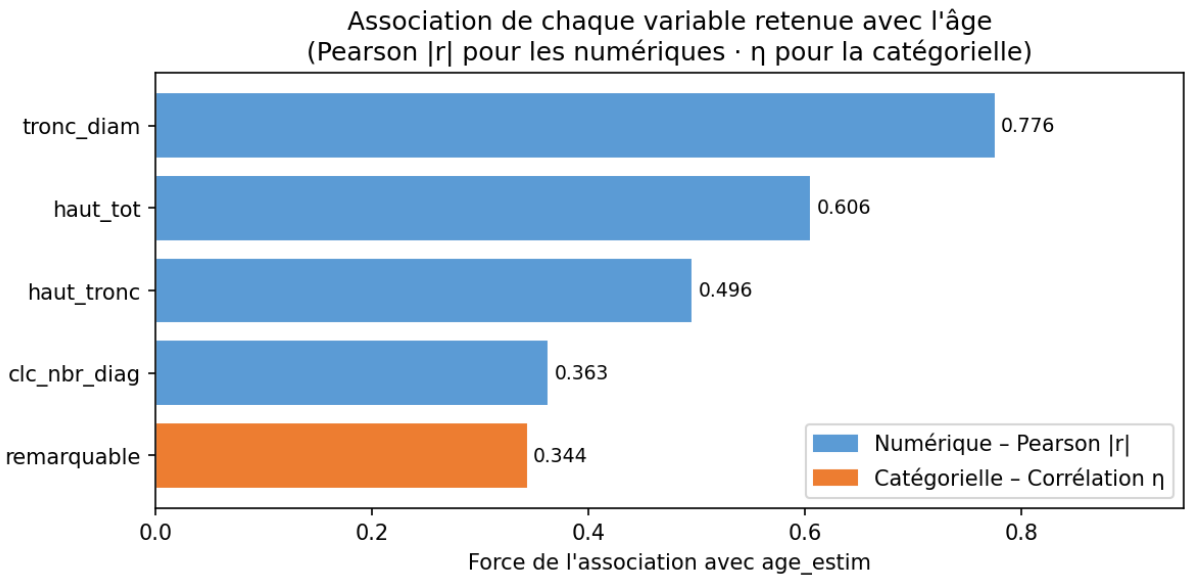
fk_situation (η = 0,159) écartée. Bien que les moyennes diffèrent légèrement selon la situation (alignement, groupe, isolé), l'association reste trop faible pour justifier l'ajout de cette variable dans le modèle.

fk_revetement (η = 0,105) écartée. La différence entre "Oui" (~41 ans) et "Non" (28 ans) semble notable, mais l'intervalle de confiance de la barre "Oui" est très large (très peu d'arbres concernés). Cette différence n'est pas robuste.

feuillage (η = 0,016) écartée. Les conifères (29,1 ans) et les feuillus (28,3 ans) ont des moyennes quasi identiques à la moyenne globale. Aucune flèche visible : cette variable n'apporte aucune information discriminante sur l'âge.

v. *Variables retenues*

Cinq variables ont été retenues sur la base de leur corrélation significative avec l'âge :



tronc_diam (numérique) : |r| = 0,776. Le diamètre du tronc est le meilleur proxy biologique de l'âge.

haut_tot (numérique) : |r| = 0,606. La hauteur totale croît avec l'âge.

haut_tronc (numérique) : |r| = 0,496. Le tronc s'allonge progressivement au fil du temps.

clc_nbr_diag (numérique) : |r| = 0,363. Les arbres âgés accumulent plus de diagnostics.

remarquable (catégoriel) : η = 0,376. Les arbres remarquables sont significativement plus âgés que les autres.

vi. Variable exclue : fk_prec_estim

Cette variable représente la précision de l'estimation d'âge renseignée par l'observateur. Elle a été exclue car elle constitue un cas de data leakage.

Le data leakage (ou fuite de données) se produit quand une variable explicative contient une information qui, en conditions réelles, ne serait pas disponible au moment de la prédiction ou qui est causalement déterminée par la cible elle-même plutôt que par les caractéristiques de l'objet à prédire. Le modèle apprend alors une relation artificielle qui n'existe que dans les données d'entraînement. Il affiche de très bonnes performances en évaluation, mais échoue complètement en production.

Dans notre cas, fk_prec_estim encode la précision avec laquelle l'observateur a estimé l'âge de l'arbre. Cette valeur est renseignée après avoir estimé l'âge : un observateur sûr de son estimation indique une précision élevée, un observateur incertain indique une précision faible. La variable est donc une conséquence de l'âge estimé, pas une cause. Sa corrélation de Pearson avec age_estim est de 0,81 plus élevée que celle du diamètre du tronc ce qui confirme qu'elle reflète directement la cible. En production, lorsqu'on veut prédire l'âge d'un arbre inconnu, cette valeur n'est pas disponible ou doit être fixée arbitrairement à une valeur par défaut, ce qui biaise toutes les prédictions vers les valeurs basses.

d) Prétraitement

Variables numériques : standardisation via StandardScaler. Cette transformation ramène chaque variable à une échelle commune : après standardisation, toutes les variables ont une moyenne de 0 et un écart-type de 1. Sans cela, tronc_diam (qui varie entre 0 et 3 m) et clc_nbr_diag (qui peut atteindre plusieurs dizaines) n'ont pas la même échelle : un modèle sensible aux distances comme Ridge accorderait mécaniquement plus de poids à la variable avec les plus grandes valeurs, indépendamment de son utilité réelle. La standardisation supprime ce biais d'échelle et permet à chaque variable de contribuer équitablement.

Point important : le StandardScaler est ajusté uniquement sur le train set, puis appliqué à la validation et au test avec les mêmes paramètres. L'ajuster sur l'ensemble du dataset ferait fuiter de l'information du test set vers le modèle ce qui est une forme de data leakage.

Variables catégorielles : encodage via OneHotEncoder. Un modèle de machine learning ne sait pas manipuler du texte : il ne peut pas interpréter "Oui" ou "Non" directement. L'encodage one-hot transforme chaque modalité en une colonne binaire (0 ou 1). Par exemple, la variable remarquable qui prend les valeurs "Oui" ou "Non" devient deux colonnes : remarquable_Oui vaut 1 si l'arbre est remarquable et 0 sinon, remarquable_Non vaut 1 dans le cas contraire. Le modèle peut alors traiter ces colonnes comme n'importe quelle variable numérique. L'option handle_unknown='ignore' permet au modèle de ne pas planter s'il rencontre une modalité inconnue lors d'une prédiction il met simplement tous les indicateurs à 0.

Découpage : le dataset est divisé en trois ensembles distincts (random_state=42). Le train set représente 60 % des données (6 184 arbres) et sert à l'entraînement des modèles. Le set de validation représente 20 % (2 062 arbres) et permet de comparer et sélectionner le modèle. Le test set représente les 20 % restants (2 062 arbres) et sert à l'évaluation finale il n'est jamais consulté avant la sélection définitive du modèle.

e) Comparaison et choix du modèle

i. Modèles candidats évalués

Trois familles de modèles de régression ont été évaluées, chacune représentant une approche différente.

Ridge Regression

La régression Ridge est une régression linéaire classique à laquelle on ajoute un mécanisme de régularisation. Une régression linéaire cherche simplement la droite (ou l'hyperplan) qui passe au plus près de tous les points du dataset. La régularisation Ridge ajoute une contrainte : les coefficients attribués aux variables ne peuvent pas devenir trop grands. Cela évite que le modèle ne s'accroche à des détails du train set qui ne se généralisent pas (overfitting).

Ce modèle est simple, rapide et très interprétable on peut lire directement l'importance de chaque variable dans ses coefficients. Sa limite est fondamentale : il suppose que la relation entre les variables et l'âge est linéaire. Or, un arbre ne vieillit pas de façon linéaire, et les interactions entre variables (par exemple, un diamètre élevé combiné au statut remarquable) ne peuvent pas être capturées par une simple droite.

Random Forest Regressor

Le Random Forest construit un grand nombre d'arbres de décision, chacun entraîné sur un sous-échantillon aléatoire du dataset et n'ayant accès qu'à un sous-ensemble aléatoire des variables à chaque nœud. La prédiction finale est la moyenne des prédictions de tous les arbres. Cette stratégie appelée bagging permet à chaque arbre d'apprendre une vision différente des données, et la moyenne lisse les erreurs individuelles.

Le Random Forest excelle pour capturer des relations complexes et non linéaires. Son problème est qu'il laisse chaque arbre croître très profondément, ce qui lui permet de mémoriser parfaitement le train set y compris son bruit. Quand on l'évalue sur de nouvelles données, cette mémorisation devient une faiblesse : le modèle a trop "appris par cœur" et ne généralise pas bien. C'est l'overfitting.

Gradient Boosting Regressor

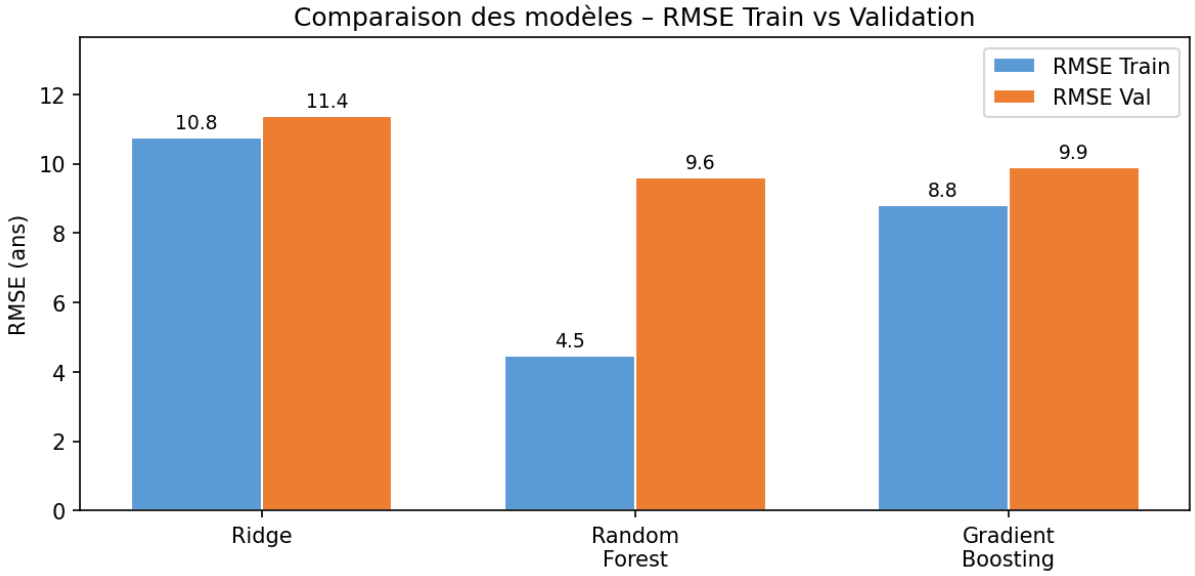
Le Gradient Boosting construit aussi des arbres de décision, mais de façon radicalement différente : les arbres sont construits séquentiellement, l'un après l'autre. Chaque nouvel arbre ne repart pas de zéro il se concentre uniquement sur les erreurs commises par l'ensemble des arbres précédents, et tente de les corriger. Avec le temps, le modèle s'améliore progressivement là où il se trompait.

Pour éviter l'overfitting, chaque arbre est volontairement peu profond (il n'apprend qu'une petite correction à la fois) et le taux d'apprentissage contrôle l'ampleur de chaque correction. Le modèle avance donc prudemment, à petits pas, ce qui lui permet de s'ajuster aux données sans les mémoriser.

ii. Pourquoi le Gradient Boosting a été retenu

Ridge est écarté car il ne peut pas modéliser les relations non linéaires entre les mesures morphologiques et l'âge ses performances sont structurellement limitées sur ce problème. Le Random Forest offre de meilleures performances brutes, mais son écart massif entre train ($R^2 = 0{,}947$) et validation ($R^2 = 0{,}785$) trahit un overfitting sévère : il a mémorisé le train set et ne généralise pas. Le Gradient Boosting présente un écart train/validation cohérent et faible, signe qu'il a réellement appris le phénomène sous-jacent plutôt que de retenir les exemples par cœur. C'est ce compromis entre puissance et généralisation qui justifie son choix.

iii. Résultats de la comparaison



Le graphique illustre l'écart entre RMSE Train et RMSE Val pour chaque modèle. Un faible écart indique une bonne généralisation (absence d'overfitting).

Ridge : RMSE de 10,76 sur le train et 11,37 sur la validation, R^2 de 0,689 et 0,698. Les deux scores sont proches (pas d'overfitting), mais le modèle sous-performe globalement. La relation âge/mesures morphologiques est trop non linéaire pour être bien capturée par une régression linéaire.

Random Forest : RMSE de 4,45 sur le train et 9,60 sur la validation, R^2 de 0,947 et 0,785. L'écart massif entre train et validation (RMSE ×2, R^2 0,947 → 0,785) est la signature d'un overfitting. Le modèle a mémorisé le bruit du train set.

Gradient Boosting : RMSE de 8,81 sur le train et 9,89 sur la validation, R^2 de 0,791 et 0,771. L'écart train/val est faible et cohérent le modèle généralise bien sans mémoriser. Il offre le meilleur compromis biais-variance des trois.

Gradient Boosting est retenu pour la phase d'optimisation.

f) Optimisation par GridSearchCV

Un modèle de machine learning a deux types de paramètres : ceux qu'il apprend à partir des données (les coefficients, les seuils des arbres...) et ceux qu'on lui fixe avant l'entraînement, appelés hyperparamètres (le nombre d'arbres, leur profondeur maximale, le taux d'apprentissage...). Ces hyperparamètres ne peuvent pas être appris automatiquement c'est au développeur de les choisir, et ils ont un impact important sur les performances.

GridSearchCV automatise cette recherche. Le principe est simple : on définit une grille de valeurs possibles pour chaque hyperparamètre, et GridSearchCV teste toutes les combinaisons possibles. Pour chaque combinaison, il évalue les performances via une validation croisée (cross-validation) : le train set est découpé en 5 parties égales (appelées folds). Le modèle est entraîné sur 4 folds et évalué sur le 5ème, et ce processus est répété 5 fois en changeant le fold d'évaluation à chaque fois. Le score final est la moyenne des 5 évaluations. Cela garantit que le choix des hyperparamètres n'est pas lié à une partition particulière des données.

Le Gradient Boosting a été optimisé sur les hyperparamètres suivants :

- **n_estimators (100, 300) :** nombre d'arbres dans l'ensemble. Plus d'arbres réduisent la variance mais augmentent le temps de calcul.
- **max_depth (3, 5, 7) :** profondeur maximale de chaque arbre. Une profondeur faible donne des arbres peu expressifs (biais élevé) ; une profondeur élevée risque l'overfitting.
- **learning_rate (0,05 ; 0,1) :** taux d'apprentissage. Un taux faible nécessite plus d'arbres mais produit un modèle plus robuste.
- **subsample (0,8 ; 1,0) :** fraction du train set utilisée pour chaque arbre. Un sous-échantillonnage à 0,8 introduit de la stochasticité et réduit l'overfitting (stochastic gradient boosting).

Meilleurs hyperparamètres : learning_rate=0.05, max_depth=5, n_estimators=300, subsample=0.8

La validation croisée (5 folds) garantit que ces hyperparamètres ont été choisis sur la base d'une estimation stable des performances, pas d'une fluctuation aléatoire.

g) Résultats du modèle final

i. Métriques sur les trois ensembles

Trois métriques sont utilisées pour évaluer les performances du modèle :

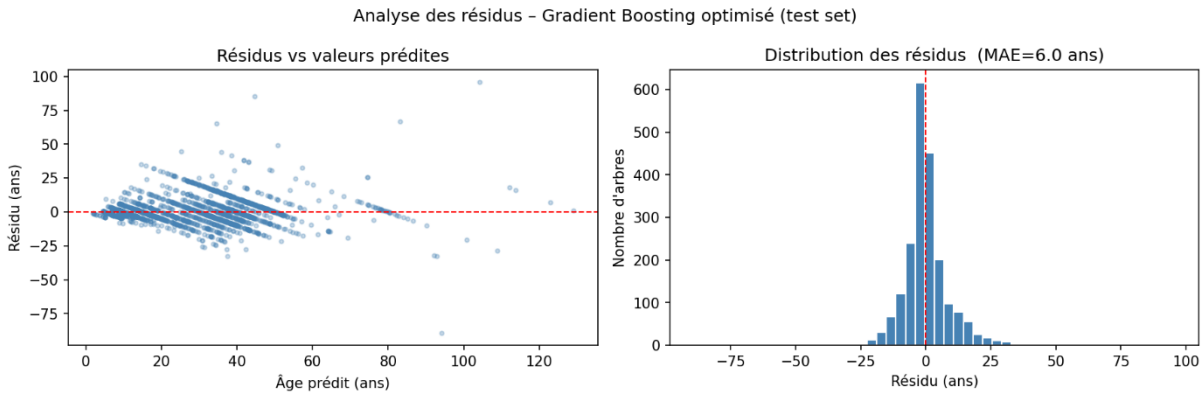
MAE (Mean Absolute Error ou Erreur Absolue Moyenne) : c'est la moyenne des écarts absolus entre l'âge prédit et l'âge réel. C'est la métrique la plus directement interprétable : une MAE de 6 ans signifie que le modèle se trompe en moyenne de 6 ans. Tous les écarts comptent de la même façon, qu'il s'agisse d'une erreur de 2 ans ou de 20 ans.

RMSE (Root Mean Square Error ou Racine de l'Erreur Quadratique Moyenne) : similaire à la MAE, mais les erreurs sont élevées au carré avant d'être moyennées. Cela a pour effet de pénaliser davantage les grandes erreurs : une erreur de 20 ans pèse beaucoup plus qu'une erreur de 2 ans. Le RMSE est donc plus sensible aux cas difficiles (arbres très anciens mal prédits). Il est toujours supérieur ou égal à la MAE un écart important entre les deux indique la présence de quelques prédictions très mauvaises.

R^2 (Coefficient de détermination) : il mesure la proportion de la variance de l'âge que le modèle parvient à expliquer. Un R^2 de 1 signifie que le modèle prédit parfaitement tous les âges. Un R^2 de 0 signifie que le modèle n'est pas meilleur que de prédire systématiquement la moyenne de l'âge pour tout le monde. En pratique, plus R^2 est proche de 1, meilleur est le modèle.

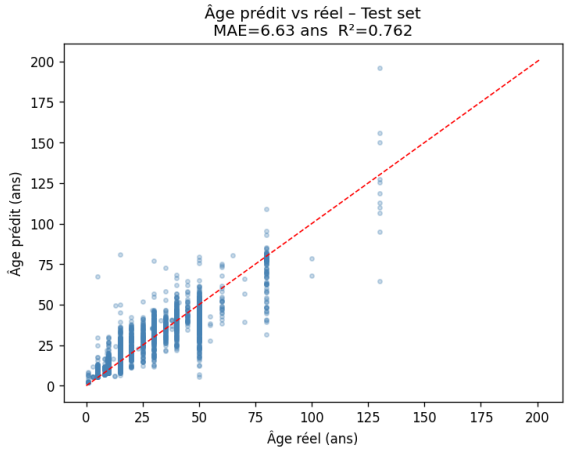
Le modèle final présente des performances cohérentes entre les ensembles. Sur le train set : MAE de 5,97 ans, RMSE de 8,49 ans, R^2 de 0,823. Sur la validation : MAE de 6,95 ans, RMSE de 10,27 ans, R^2 de 0,769. Sur le test set (évaluation finale) : MAE de 6,63 ans, RMSE de 9,73 ans, R^2 de 0,762. La proximité entre validation (10,27) et test (9,73) confirme l'absence d'overfitting. Le modèle se trompe en moyenne de 6,6 ans sur l'estimation de l'âge.

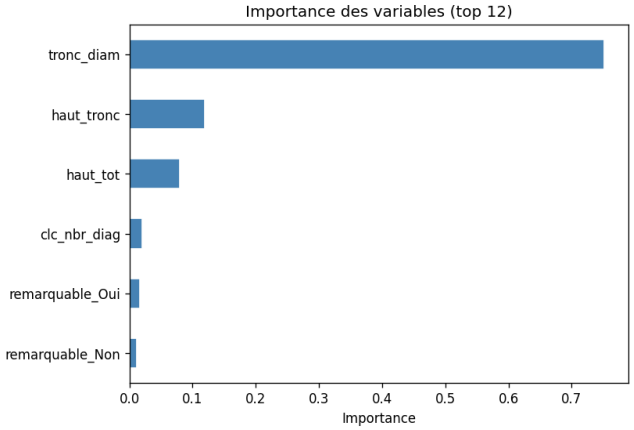
ii. Analyse des résidus



Le graphique de gauche (résidus vs valeurs prédites) permet de détecter des biais systématiques. Une bonne régression produit des résidus répartis aléatoirement autour de zéro, sans structure. Le graphique de droite montre la distribution des résidus : elle est centrée sur zéro et approximativement symétrique, ce qui indique que le modèle ne surestime ni ne sous-estime systématiquement l'âge. La légère asymétrie vers les valeurs négatives s'explique par la difficulté à prédire les arbres très anciens (rares dans le dataset).

iii. Prédiction vs réel et importance des variables





Le nuage de points "prédit vs réel" montre l'alignement des prédictions sur la diagonale idéale (ligne rouge). La dispersion augmente pour les âges élevés (>80 ans), ce qui est attendu car ces arbres sont peu représentés. Le graphique d'importance des variables confirme que tronc_diam est de loin la variable la plus discriminante, suivi de haut_tot et haut_tronc, cohérent avec l'analyse de corrélation initiale.

h) Conclusion

Le modèle de Gradient Boosting Regressor optimisé atteint un R^2 de 0,762 et une MAE de 6,6 ans sur le test set, ce qui est satisfaisant compte tenu du fait que les âges dans la base de données sont eux-mêmes des estimations approximatives.

Seules 5 variables ont été retenues après analyse de leur corrélation avec l'âge : les 4 variables numériques morphologiques et remarquable, la seule variable catégorielle présentant une association significative ($\eta = 0{,}376$). Les variables feuillage, fk_situation et fk_revetement ont été écartées en raison d'une corrélation insuffisante avec l'âge, confirmée par l'analyse du rapport de corrélation η.

Le choix du Gradient Boosting est justifié par sa capacité à capturer des relations non linéaires (avantage sur Ridge) et sa résistance naturelle à l'overfitting grâce à l'apprentissage séquentiel corrigé (avantage sur Random Forest). La principale limite du modèle est sa précision sur les arbres très anciens, peu représentés dans le dataset, pour lesquels l'erreur de prédiction tend à être plus élevée.

IV. Besoin Client 3 : Système d'alerte pour les tempêtes (Romann)

Le besoin du client est d'avoir un système d'alerte pour détecter si un arbre est susceptible de tomber en cas de tempête. Pour cela, il suffit d'entraîner une IA à partir des données étudiées précédemment pour déterminer si oui ou non un arbre est en danger.

Le cahier des charges nous propose d'utiliser la statistique liée à l'état de l'arbre. Néanmoins, cette donnée ne précise pas si un arbre a été abattu à cause d'une tempête ou à cause d'un autre facteur.

Ce point va nous obliger à établir nous-même une condition pour savoir si un arbre est vraiment en danger ou non. A partir de cela, l'IA sera en mesure d'apprendre les conditions nécessaires pour définir si un arbre est à risque.

a) Sélection des données utiles

Parmi les données précédemment nettoyées, seulement quelques colonnes sont susceptibles d'être utiles pour étudier si un arbre est sensible à tomber lors d'une tempête. Pour les déterminer, il suffit de s'intéresser à chaque donnée du CSV et déduire si elle est utile ou non.

Données retenues

- Morphologie / taille :
  - haut_tot : hauteur totale
  - haut_tronc : hauteur du tronc
  - tronc_diam : diamètre du tronc

  Justification : plus un arbre est haut, massif ou avec certaines proportions, plus son comportement face au vent peut changer.

- Âge / développement :
  - age_estim : Estimation de l'âge de l'arbre
  - fk_stadedev : Stade de développement de l'arbre

  Justification : un arbre jeune, adulte, vieux ou sénescent n'a pas la même résistance.

- Contexte de plantation / ancrage :
  - fk_pied
  - fk_revetement
  - fk_situation

  Justification :

  - un arbre dans du **gazon / terre** n'a pas le même enracinement qu'en **fosse arbre** ou en zone plus contrainte ;
  - fk_revetement : peut aider à approcher la contrainte du sol ;
  - fk_situation (isolé, groupe, alignement) : peut influencer l'exposition au vent

- Architecture de l'arbre :
  - fk_port

  Justification : la forme du port modifie la prise au vent.

- Essence / espèce :
  - feuillage

  Justification : certains types de feuillage peuvent être plus sensibles au vent ou à certains types de rupture.

b) Définir le paramètre « alerte »

Pour construire un référentiel exploitable par l'IA, il faut définir les conditions pour dire si un arbre est en danger ou non. Pour ce faire, on peut établir un système de score. Plus ce score va être élevé, plus le risque de déracinement est élevé.

Exemple pour des variables numériques :

- Hauteur de l'arbre :
  - Si la hauteur totale de l'arbre est supérieure à la hauteur médiane, le score prend +1.
  - Si la hauteur totale de l'arbre est supérieure à la valeur du quartile 3, le score prend +1.
- Diamètre du tronc:
  - Si le diamètre du tronc est inférieur au diamètre médian, le score prend +1.
  - Si le diamètre du tronc est inférieur au à la valeur du quartile 1, le score prend +1.
- …

Exemple pour des variables textuelles :

- Stade de développement :
  - Si l'arbre est jeune : Pas d'impact sur le score
  - Si l'arbre est adulte : +1 sur le score
  - Si l'arbre est vieux : +2 sur le score
  - Si l'arbre est sénescent : +3 sur le score
  - Sinon : Pas d'impact sur le score
- Situation de l'arbre :
  - Si l'arbre est en groupe : Pas d'impact sur le score
  - Si l'arbre est aligné avec d'autres arbres : +1 sur le score
  - Si l'arbre est isolé : +2 sur le score
  - Sinon : Pas d'impact sur le score
- …

Maintenant qu'on a établi un score pour chaque arbre, on peut définir un seuil de score qui déterminera si l'arbre est en danger ou non.

Ici le score seuil est de : **7**

Cela signifie que si le score est supérieur à 7, alors l'arbre est considéré comme en danger.

Après analyse, la classification donne :

- 93% (10714 arbres) : Arbres ne représentant aucun danger
- 7% (704 arbres) : Arbres en danger

c) Sélection du meilleur modèle d'apprentissage

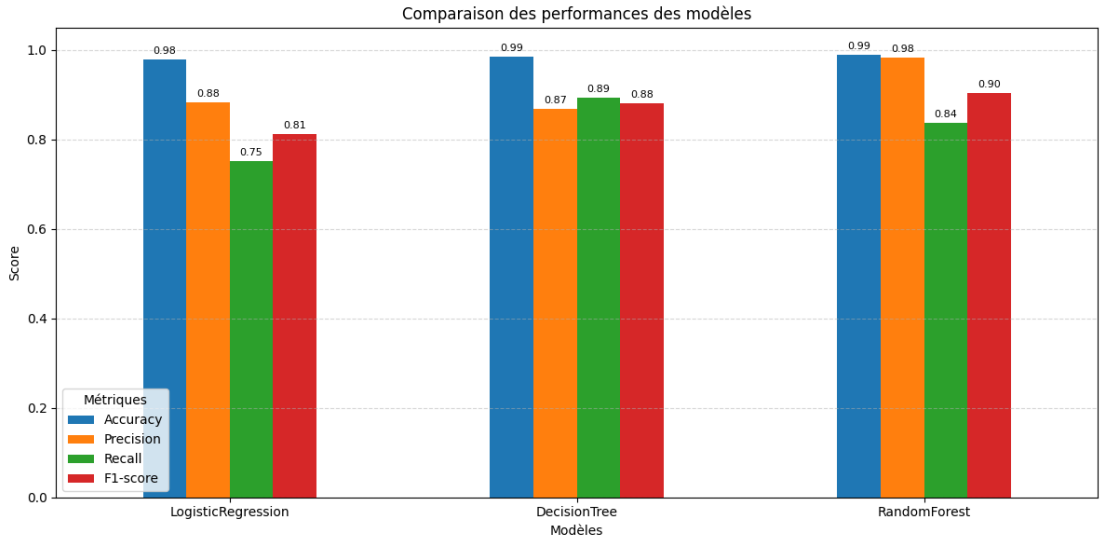
Notre base étant prête, notre modèle est maintenant prêt à apprendre en utilisant nos données. Pour réaliser le meilleur apprentissage possible, nous allons d'abord étudier quel est le meilleur modèle à utiliser.

Les trois modèles qui vont être testés sont :

- LogisticRegression
- DecisionTree
- RandomForest

Pour tester ces algorithmes, nous allons établir une base d'entrainement (80% du dataset) et une base de test (20% du dataset).

On obtient les résultats suivants :



Le RandomForest est donc le meilleur modèle parmi les 3 car il possède une précision et une accuracy très élevée. Néanmoins sont score de Recall est un peu bas mais il pourra être optimisé par la suite.

d) Entrainement du modèle

i. Prétraitement des données

Afin de faciliter l'apprentissage de notre modèle, nous allons séparer le dataset en deux parties. Une partie contenant les variables d'entrées mentionnées ci-dessus et une partie contenant ce que le modèle doit prédire (colonne « alerte » créée précédemment).

Pour préparer allons le séparer en une base d'entraînement (80% du dataset) et une base de test (20% du dataset). Cela va nous permettre d'évaluer la qualité de notre modèle après apprentissage en utilisant notre base de test.

Enfin, les variables catégorielles sont encodées avec **OneHotEncoder**. Cette méthode convertit les valeurs textuelles en colonnes numériques binaires pour que l'algorithme puisse les traiter correctement.

ii. Optimisation des données

On cherche à obtenir la meilleure configuration possible pour notre modèle. Pour la trouver, nous allons utiliser **GridSearchCV.** Celui-ci va trouver les meilleurs hyperparamètres possible pour notre modèle (nombres d'arbres, profondeur maximale de l'arbre, …) en testant toutes les combinaisons possibles. Cela va beaucoup jouer sur la qualité des prédictions et éviter le risque de sur-apprentissage. Il va également utiliser la validation croisée en choisissant 5 folds (5 sous-parties de notre dataset), puis va entrainer notre modèle 5 fois en utilisant des folds différents pour l'apprentissage et le test à chaque fois. Cela va nous permettre d'obtenir une évaluation de notre modèle bien plus fiable que si on le faisait seulement 1 seule fois au hasard.

iii. Entrainement du modèle

Comme déterminé précédemment, le modèle que nous allons utiliser est **RandomForest**. Il repose sur la création de plusieurs arbres de décision entraînés sur des échantillons aléatoires de notre dataset. Lorsqu'une nouvelle observation est fournie, chaque arbre émet une prédiction, puis le modèle retient la classe majoritaire. Cette méthode améliore généralement la stabilité et la capacité de généralisation par rapport à un arbre de décision seul.

iv. Evaluation du modèle

Après apprentissage, il est important d'évaluer la qualité de notre modèle. Pour cela plusieurs variables nous indiquent la qualité de notre modèle :

- **L'accuracy** : Mesure la proportion totale de bonnes prédictions réalisées par le modèle
- **La précision :** Indique, parmi les arbres prédits comme étant en alerte, la proportion de ceux qui sont effectivement à risque
- **Le recall :** Mesure, parmi les arbres réellement en alerte, la proportion de ceux qui ont bien été détectés par le modèle
- **Le F1-score** : Correspond à la moyenne harmonique entre la precision et le recall

Dans notre cas, on obtient les résultats suivants (pour la base de test) :

Accuracy	**Precision**	**Recall**	**F1-score**
0.98992	0.98360	0.85106	0.91254

On voit que notre modèle obtient de très bonnes performances globales. Néanmoins le recall reste faible par rapport aux autres performances, ce qui veut dire que notre modèle détecte une grande partie des arbres considérés comme à risque mais qu'il reste quelques oublis.

Afin de vérifier que notre modèle ne fait pas de sur-apprentissage, nous pouvons comparer la performance sur le jeu de test et sur le jeu d'apprentissage.

Résultats sur la base d'apprentissage :

Accuracy	**Precision**	**Recall**	**F1-score**
0.99912	0.98598	1.0	0.99294

On remarque de l'écart entre les tests sur la base d'apprentissage et la base de tests sont relativement similaires, ce qui prouve que notre modèle ne fait pas de sur-apprentissage tout en gardant une grande précision sur les valeurs prédites.

v. *Script de prédiction d'alerte*

Afin de prédire si un arbre est en alerte ou non, nous allons utiliser un script python se basant sur notre modèle (extrait en .pkl). Ce script prend en entrée les caractéristiques de l'arbre et renvoie si oui ou non l'arbre est en alerte et la probabilité qu'il soit en risque.

Voici un exemple pour un arbre avec les caractéristiques suivantes :

- Hauteur totale : 12,5m
- Hauteur du tronc : 3,2m
- Diamètre du tronc : 0,45m
- Age estimé : 30 ans
- Stade de développement : adulte
- Forme du port : Rideau
- Type de sol : gazon
- Situation : Alignement (aligné avec d'autres arbres)
- Revêtement : Non
- Feuillage : Feuillu

Résultat du script :

```
● (.venv) isen@LAPTOP-OGNG4RK1:~/BDD-IA-WEB/IA/3-Systeme-alerte-tempête$ python predire_alerte.py \
          --haut_tot 12.5 \
          --haut_tronc 3.2 \
          --tronc_diam 0.45 \
          --age_estim 30 \
          --fk_stadedev adulte \
          --fk_port rideau \
          --fk_pied gazon \
          --fk_situation alignement \
          --fk_revetement non \
          --feuillage feuillu
  === Résultat de la prédiction ===
   haut_tot  haut_tronc  tronc_diam  age_estim fk_stadedev fk_port fk_pied fk_situation fk_revetement feuillage
       12.5         3.2        0.45       30.0      adulte  rideau   gazon   alignement           non   feuillu

  PAS D'ALERTE : cet arbre est considéré comme peu susceptible d'être déraciné en cas de tempête.
  Probabilité estimée de risque : 0.0421
```